Laboratory 6

Image Classification with Neural Network

Mohsen Pourghasemian, Sadaf Joodaki.
emml@dsp.rwth-aachen.de

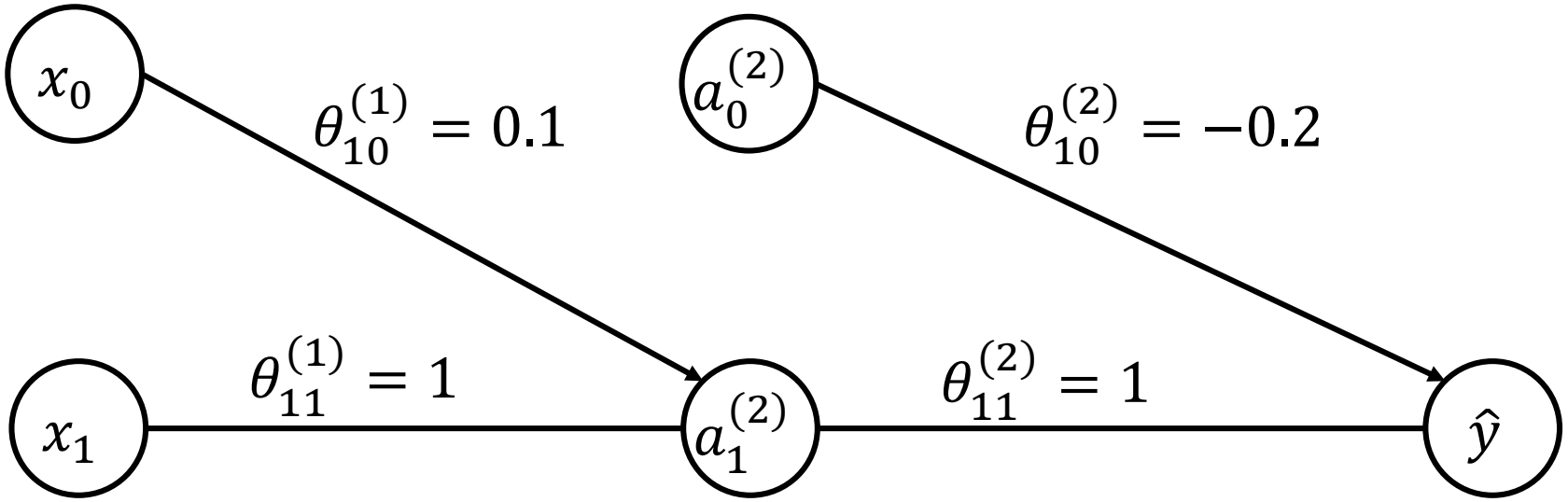
Overview of exercises

- Laboratory 1: K-Means Clustering (1.0 Point)
- Laboratory 2: Grid Search for Linear Regression (1.0 Point)
- Laboratory 3: Gradient Descent for Linear Regression (1.5 Point)
- Laboratory 4: Logistic Regression (1.0 Point)
- Laboratory 5: Basics of Neural Network (1.0 Point)
- Laboratory 6: Image Classification with Neural Network (1.5 Points)
- Laboratory 7: Reinforcement learning (1.5 Point)
- Laboratory 8: Deep RL (1.5 Point)

Training a Neural Network with Two Training Dataset using BGD

Neural network architecture

- Consider a neural network with the following architecture:



Training a neural network with two training examples

- Consider a neural network with the following setting:
  - The activation function for both the hidden and output layer is “sigmoid”.
  - The learning rate is $\alpha = 0.1$
  - Consider the cost function as

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

- Perform one step training for the network and write the training process step by step.

Training dataset

- The training dataset is given in the table below

Training dataset

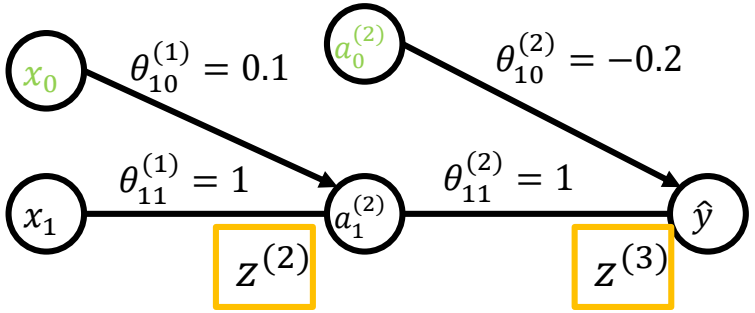
$i = 1$	$i = 2$
$x = 0.5$	$x = 0.2$
$y = 0.3$	$y = 0.2$

Forward Propagation

Feedforward for training example 1 ($i = 1$)

- Calculate the values of each neuron for the first training example:

$$z^{(2)} = \theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 = 0.1 \times 1 + 1 \times 0.5 = 0.6$$
$$a_1^{(2)} = g\left(z^{(2)}\right) = \frac{1}{1+e^{-(z^{(2)})}} = \frac{1}{1+e^{-0.6}} = 0.6457$$

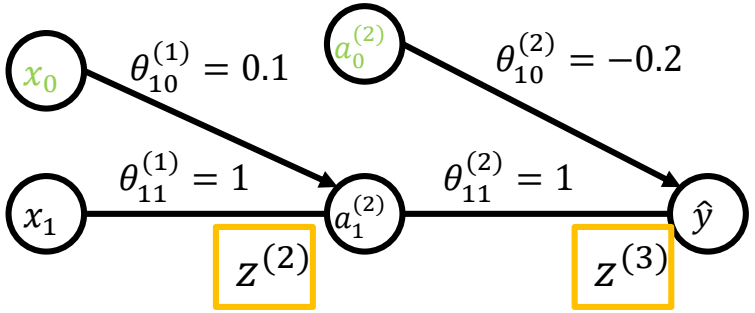


$$z^{(3)} = \theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} = -0.2 \times 1 + 1 \times 0.6457 = 0.4457$$
$$\hat{y} = g\left(z^{(3)}\right) = \frac{1}{1+e^{-(z^{(3)})}} = \frac{1}{1+e^{-0.4457}} = 0.6095$$

Feedforward for training example 2 ($i = 2$)

- Calculate the values of each neuron for the second training example:

$$z^{(2)} = \theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 = 0.1 \times 1 + 1 \times 0.2 = 0.3$$
$$a_1^{(2)} = g\left(z^{(2)}\right) = \frac{1}{1+e^{-(z^{(2)})}} = \frac{1}{1+e^{-0.3}} = 0.574$$



$$z^{(3)} = \theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} = -0.2 \times 1 + 1 \times 0.574 = 0.374$$
$$\hat{y} = g\left(z^{(3)}\right) = \frac{1}{1+e^{-(z^{(3)})}} = \frac{1}{1+e^{-0.374}} = 0.592$$

Cost for both training examples

- Calculating the total cost using MSE:

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i}\left(\hat{y}^{(i)} - y^{(i)}\right)^2 = \frac{1}{2m}\left[\left(\hat{y}^{(1)} - y^{(1)}\right)^2 + \left(\hat{y}^{(2)} - y^{(2)}\right)^2\right]$$

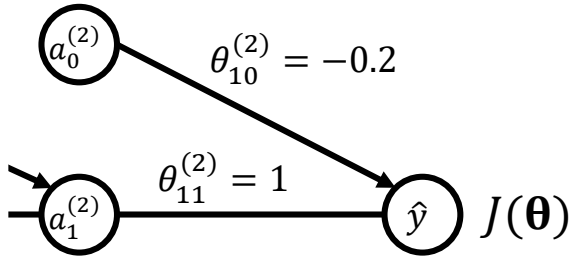
$$= \frac{1}{2 \times 2}\left((0.6095 - 0.3)^2 + (0.5925 - 0.2)^2\right) = 0.0626$$

Back Propagation

Back propagation for output ($i = 1$)

- Calculate the back propagation values of output for the first training example (for $\theta_{11}^{(2)}$):

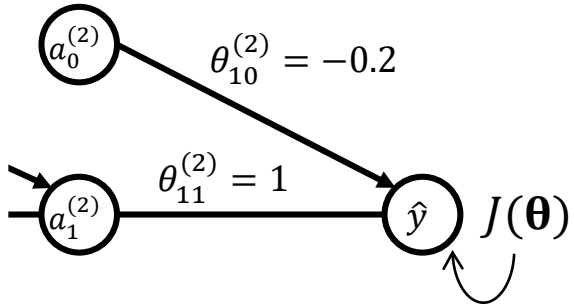
$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = ?$$



Back propagation for output ($i = 1$)

- Calculate the back propagation values of output for the first training example (for $\theta_{11}^{(2)}$):

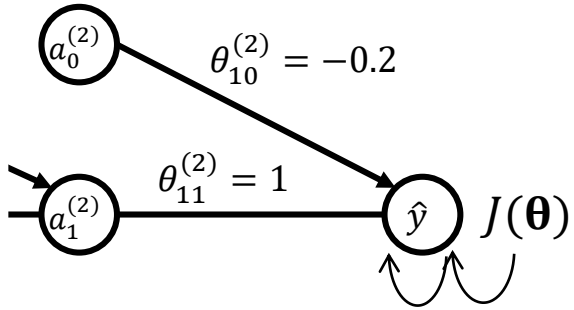
$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}}$$



Back propagation for output ($i = 1$)

- Calculate the back propagation values of output for the first training example (for $\theta_{11}^{(2)}$):

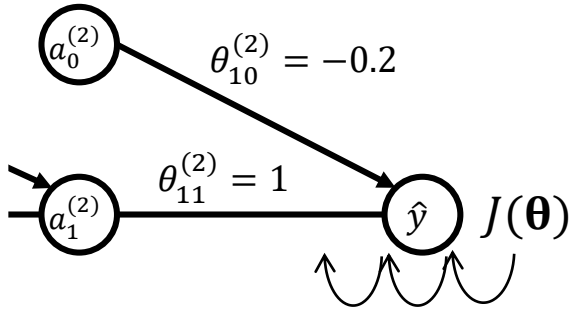
$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(3)}}$$



Back propagation for output ($i = 1$)

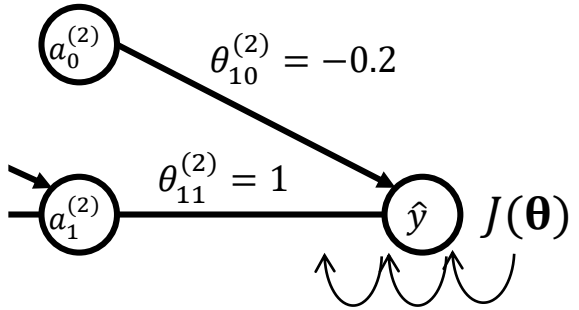
- Calculate the back propagation values of output for the first training example (for $\theta_{11}^{(2)}$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial \theta_{11}^{(2)}}$$



Back propagation for output ($i = 1$)

- Calculate the back propagation values of output for the first training example (for $\theta_{11}^{(2)}$):



$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial \theta_{11}^{(2)}}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} = \hat{y} - y = 0.6095 - 0.3 = 0.3095$$

$$\frac{\partial \hat{y}}{\partial z^{(3)}} = \hat{y}(1 - \hat{y}) = 0.238$$

$$\frac{\partial z^{(3)}}{\partial \theta^{(2)}} = a_1^{(2)} = 0.6457$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = 0.3095 \times 0.238 \times 0.6457 = 0.0476$$

Back propagation for output ($i = 1$)

- Calculate the back propagation values of output for the first training example (for $\theta_{10}^{(2)}$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial \theta_{10}^{(2)}} = 0.3095 \times 0.238 \times 1 = 0.0737$$

Back propagation to hidden layer ($i = 1$)

- Calculate the back propagation values of hidden layer for the first training example (for $\theta_{11}^{(1)}$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial \theta_{11}^{(1)}}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial a^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(3)}} \cdot \frac{\partial z^{(3)}}{\partial a^{(2)}} = 0.3095 \times 0.238 \times 1 = 0.0737$$

$$\frac{\partial a^{(2)}}{\partial z^{(2)}} = a^{(2)}\left(1 - a^{(2)}\right) = 0.2288$$

$$\frac{\partial z^{(2)}}{\partial \theta_{11}^{(1)}} = x = 0.5$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = 0.0737 \times 0.2288 \times 0.5 = 0.00845$$

Back propagation to hidden layer ($i = 1$)

- Calculate the back propagation values of hidden layer for the first training example (for $\theta_{10}^{(1)}$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = +0.0169$$

Back propagation to hidden layer ($i = 1$)

- Calculate the back propagation values of hidden layer for the first training example (for $\theta_{10}^{(1)}$):

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = +0.0169$$

You can calculate for the second training example (for $i = 2$)

Gradients for both training examples

- Then the values for first and second Training examples are:

$i = 1$	$i = 2$
$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = +0.0476$	$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = +0.0544$
$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(2)}} = +0.0737$	$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(2)}} = +0.0947$
$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = +0.00845$	$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = +0.00462$
$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = +0.0169$	$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = +0.0231$

Final Gradients

$$i = 1 \qquad i = 2$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = \frac{1}{2}(0.0476 + 0.0544) = +0.0510$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = \frac{1}{2}(0.00845 + 0.00462) = +0.00654$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(2)}} = \frac{1}{2}(0.0737 + 0.0947) = +0.0842$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = \frac{1}{2}(0.0169 + 0.0231) = +0.0200$$

Theta parameter updates

$$\theta_{11}^{(1)} \leftarrow \theta_{11}^{(1)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(1)}} = 0.9993$$

$$\theta_{11}^{(2)} \leftarrow \theta_{11}^{(2)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{11}^{(2)}} = 0.9949$$

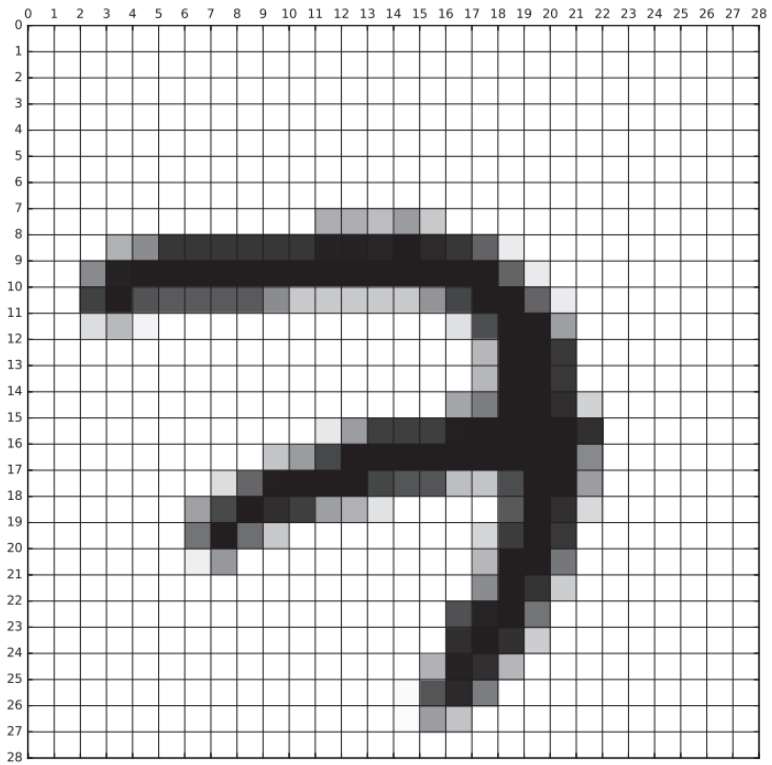
$$\theta_{10}^{(1)} \leftarrow \theta_{10}^{(1)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(1)}} = 0.0980$$

$$\theta_{10}^{(2)} \leftarrow \theta_{10}^{(2)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_{10}^{(2)}} = -0.2084$$

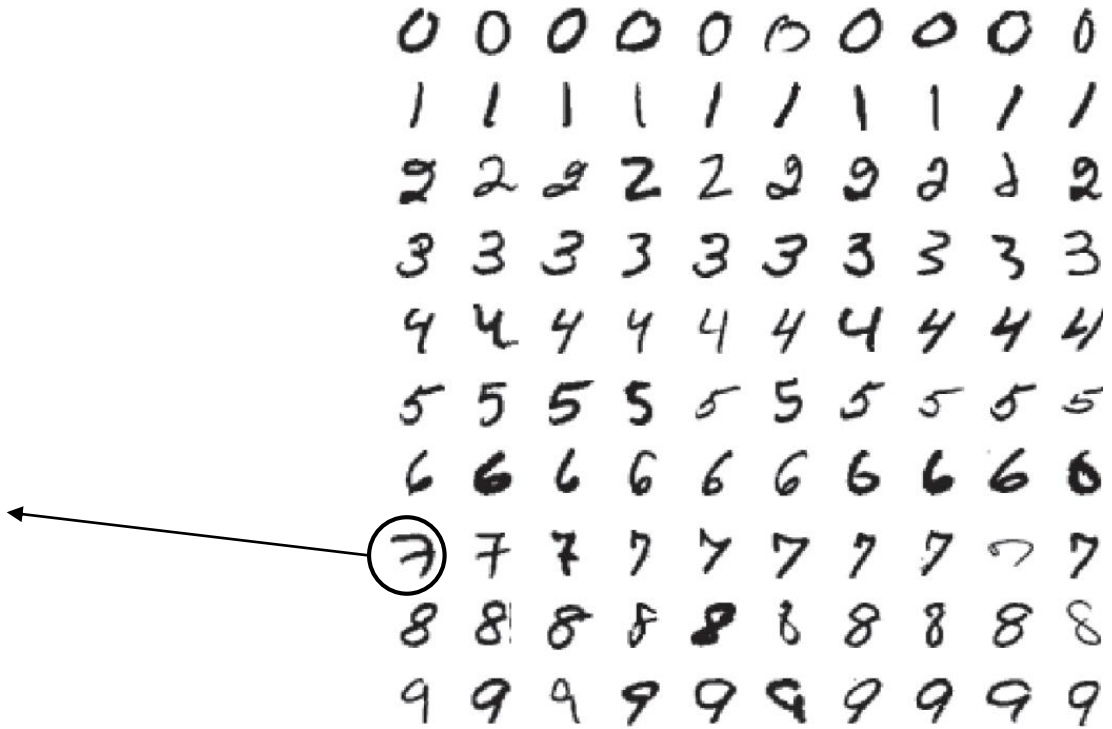
Assignment

MNIST handwritten digits dataset

- The MNIST database is the Modified National Institute of Standards and Technology.
- It includes 70000 handwritten digits, each is an image with the size of 28x28 pixels.



The image belongs to number 7 (28x28 pixels)



100 samples from the MNIST training set

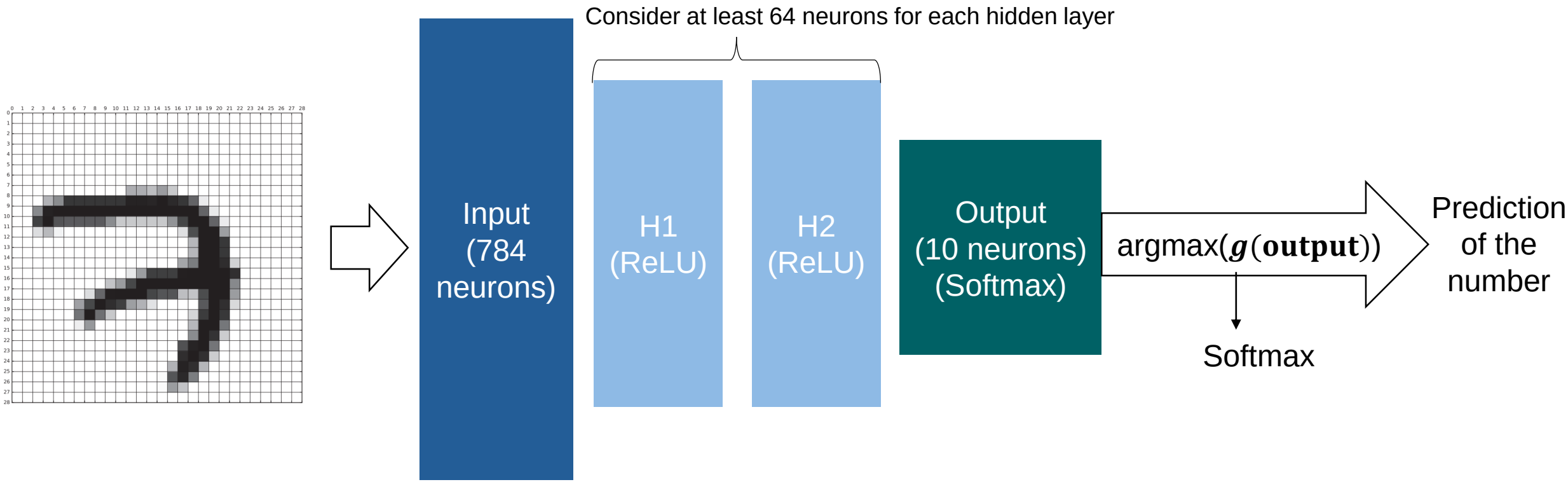
[1] http://yann.lecun.com/exdb/mnist/index.html

Objective

- **Objective:** Implement a Neural Network to classify MNIST Handwritten digits. The neural network architecture is provided.
  - You should not use machine learning libraries like Tensorflow, Pytorch etc. for creation/training the neural network.
  - Use the "*E6_Student_code.py*" and fill the code where required.

Neural network architecture for this assignment

- The neural network architecture is assumed as following:



One example structure of a neural network for MNIST handwritten digit classification

Tasks

Step 1. Implement activation functions, with corresponding functions as

- *relu(Z)*
- *relu_derivative(Z)*
- *softmax(Z)*

Step 2. Implement forward propagation:

- Compute all necessary intermediate values.
- The corresponding function is *forward(X, Theta1, Theta2, Theta3)*.

Tasks, contd.

Step 3. Compute the cost using the cross-entropy (log-loss) function with

- *cross_entropy_cost(y_hat, Y_true)*

Step 4. Implement backpropagation with

- *backward(X, Y_true, Theta1, Theta2, Theta3)*

Step 6. Update weights and biases using gradient descent

- Perform the parameter updates inside the training loop.
- This update step should be written inside the for loop that iterates over the epochs:
  for epoch in range(epochs) …

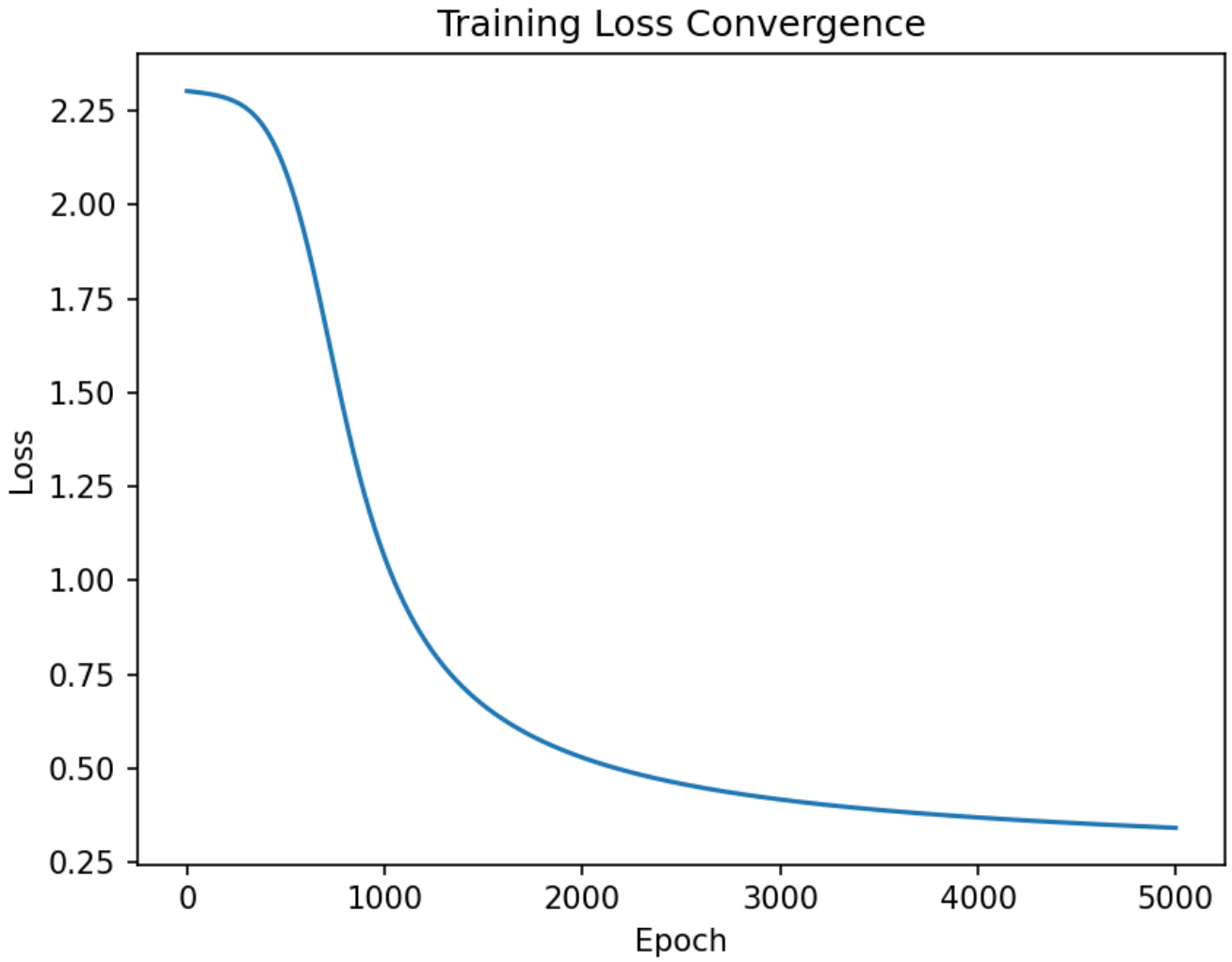
Submission

- Submission Requirement
  - Submit your .py file that produces the expected results as specified in the assignment.

- Submit your code on Moodle by **June 15, 2026, at 10:30 AM**.
- Do not import any additional libraries or modules that are not already included in the provided code.

❖ If the code fails to run, it will receive no points.

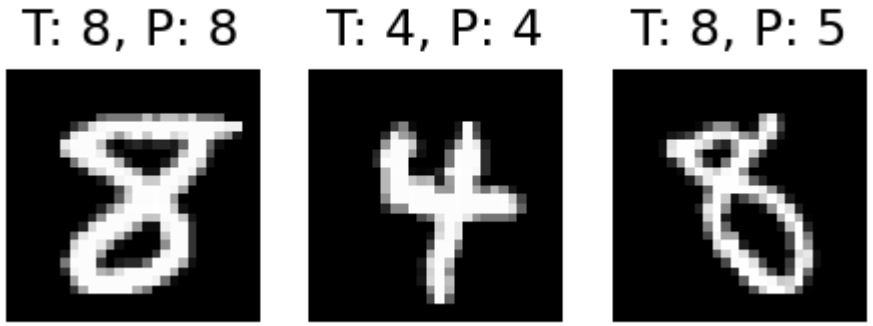
Expected results, training loss convergence



Expected results, print the accuracy test

```
Epoch 93 / 100 cost = 0.3986
Epoch 94 / 100 cost = 0.3969
Epoch 95 / 100 cost = 0.3953
Epoch 96 / 100 cost = 0.3936
Epoch 97 / 100 cost = 0.392
Epoch 98 / 100 cost = 0.3905
Epoch 99 / 100 cost = 0.3889
Epoch 100 / 100 cost = 0.3874
Test Accuracy = 89.02 %
```

Expected results, displazing a few test images



T: Input Test, **P**: Predicted

Additional notes

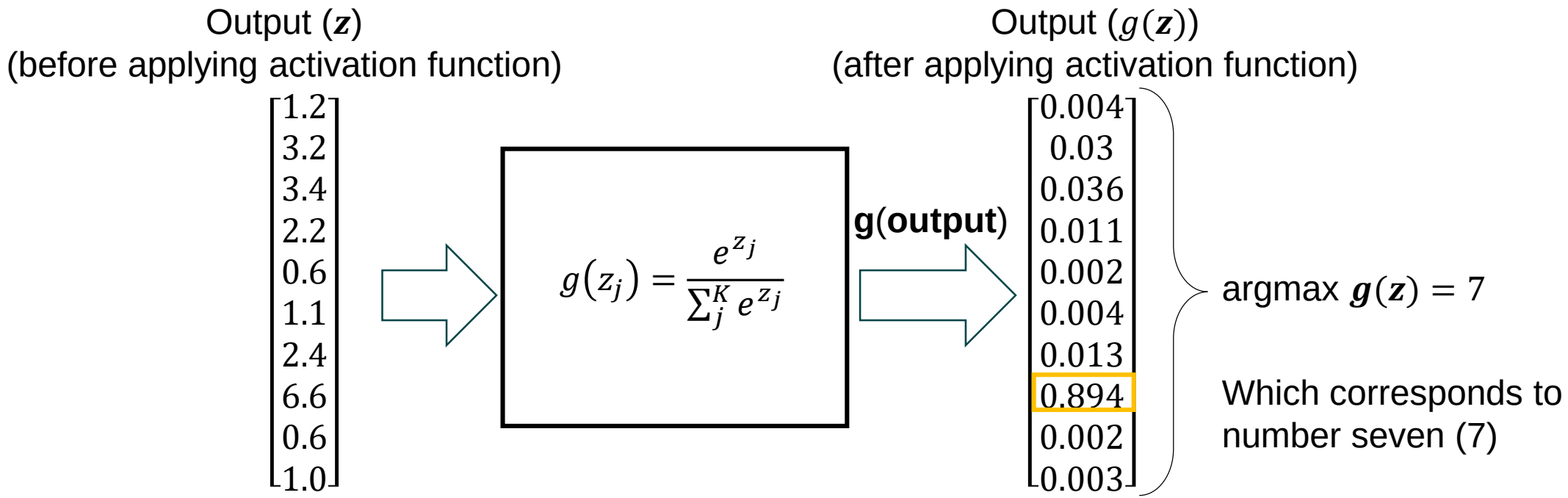
Additional Notes about the activation functions

- Note1: The activation function for hidden layers is ReLu:

$$g(z) = \max(0, z)$$

- Note2: The activation function for output layer is Softmax:

$$g(z_j) = \frac{e^{z_j}}{\sum_j^K e^{z_j}}$$



Additional Notes about the activation functions

- The i-th output $\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$, where $\boldsymbol{z} = [z_1, \dots, z_K]$
- With softmax, we usually use cross-entropy loss
  - $J(\theta) = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$, where $\boldsymbol{y} = [y_1, \dots, y_K]$ is the one-hot encoded true label.

1. Now we calculate the loss with respect to softmax output :
   - $\frac{\partial J(\theta)}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$
2. We need to calculate the gradient of softmax output with respect to input z_k [1]:
   - $\frac{\partial \hat{y}_i}{\partial z_k} = \hat{y}_i(\delta_{ik} - \hat{y}_k), \qquad \delta_{ik} \begin{cases} 1 & i = k \\ 0 & i \neq k \end{cases}$
3. And now the chain rule:
   - $\frac{\partial J(\theta)}{\partial z_k} = \sum_{i=1}^{K} \frac{\partial J(\theta)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_k} = \sum_{i=1}^{K} \left(-\frac{y_i}{\hat{y}_i} \cdot \hat{y}_i(\delta_{ik} - \hat{y}_k) \right) = -\sum_{i=1}^{K} y_i(\delta_{ik} - \hat{y}_k)$

[1] https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/

Additional Notes about the activation functions

Since $y_i = 0$ for $i \neq k$, and $y_k = 1$:

- $\frac{\partial J(\theta)}{\partial z_k} = \hat{y}_k - y_k$

Additional Notes about back propagation

Since $y_i = 0$ for $i \neq k$, and $y_k = 1$:

- $\frac{\partial J(\theta)}{\partial z_k} = \hat{y}_k - y_k$

- Reminder: The sum of all outputs after activation function is 1.

$$\sum_{i=0}^{9} g(z_i) = 0.004 + 0.03 + \cdots + 0.003 = 1$$

4. After applying the activation function, you need to select the output which has the highest value among others. The argument of the maximum value corresponds to the predicted number.

End of Exercise 6

Thank you!

RWTH Aachen University | Email : pourghasemian@dsp.rwth-aachen.de | Web : www.dsp.rwth-aachen.de

DSP Chair for Distributed Signal Processing

RWTH AACHEN UNIVERSITY